

Computer Vision: Object Detection Algorithms and Image Segmentation

Agenda of this session:

- Definition of Object Detection and Image Segmentation
- Object Detection Algorithm
 - RCNN
 - FAST-RCNN
 - FASTER-RCNN
 - YOLO
- Image Segmentation
 - Semantic Segmentation
 - Instance Segmentation

Definition of Object Detection:

Classification



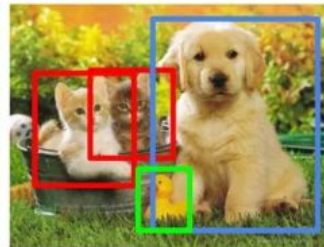
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**



CAT, DOG, DUCK

Single object

Multiple objects

Object classification

- The model looks at the entire image and assigns a label to it based on what it “sees.”
- Object classification doesn’t tell you where the object is located in the image. It only provides the label.
- Object classification assigns a single label to the entire image. If the image contains both a dog and a cat, the model will likely predict the label that it is most confident about.

Object Localization:

- Object localization goes one step further. It is not only identifies the object in the image but also determines **where** the object is located by drawing a **bounding box** around it.
- The model predicts the class of the object (e.g., cat, dog) and the coordinates of a bounding box that surrounds the object.

Limitation:

- Object localization works well when there is **only one object** in the image. It struggles when multiple objects are present.

Object Detection

Object detection combines the tasks of classification and localization but at a more advanced level. It answers two questions:

1. **What objects are in the image?**
2. **Where are they located?**

How it Works:

- The model identifies all the objects in the image, labels them, and draws bounding boxes around each one.
- Object detection can handle **multiple objects** in the same image, even if they vary in size and position.

Introduction to Object Detection

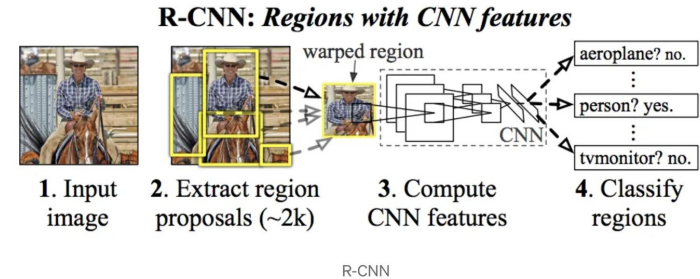
- Goal: Locate and classify objects in images.
- Key tasks:
 - Classification
 - Localization(bounding box)
- Challenges:
Scale Variation, occlusion, real-time performance

Evolution of Object Detection

- Traditional Methods: Sliding windows+classifiers
- Deep learning revolution:
 - R-CNN(2014)
 - Fast R-CNN(2015)
 - Faster R-CNN(2015)
- One-stage methods: Yolo, SSD (Covered separately)

Definition of Object Detection: R-CNN

- 1- Select the image.
- 2- Extract 2000 candidate region proposals
- 3- Pass it to the CNN network.
- 4- Convolution Neural Network produces a 4096-dimensional feature vectors as output.
- 5- The CNN act as a feature extractor and the output of dense layer consists of the features extracted from the image and the extracted features are fed into an SVM to classify the presence of the object within that candidate region proposal.



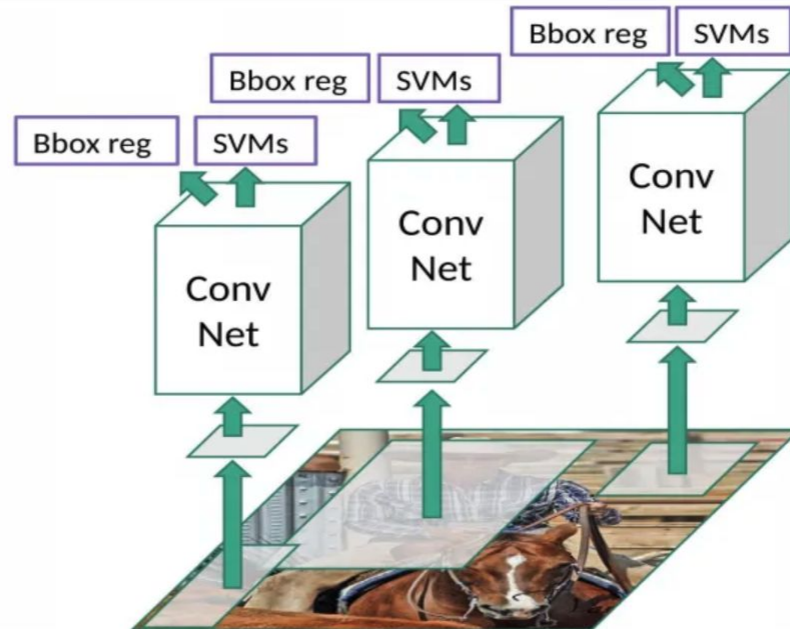
Selective Search:

- 1- Generate initial sub-segmentation, it generates many candidate regions
- 2- Use greedy algorithm to recursively combine similar regions into larger ones.
- 3- Use the generated regions to produce the final candidate region proposals.

R-CNN(Region-based CNN)

- **Step-by-step:**
 - Use selective search to extract ~2K region proposals
 - Warp each region to fixed size
 - Run CNN (AlexNet) on each region
 - Extract features, classify with SVM, refine boxes with regressor
- **Advantages:**
 - High accuracy
- **Drawbacks:**
 - Slow: ~47s/image (no GPU)
 - No shared computation

R-CNN



R-CNN

RCNN Architecture Diagram:

- Input Image
- Selective Search
- Region Proposals
- CNN Feature Extractor(per region)
- SVM+Regressor

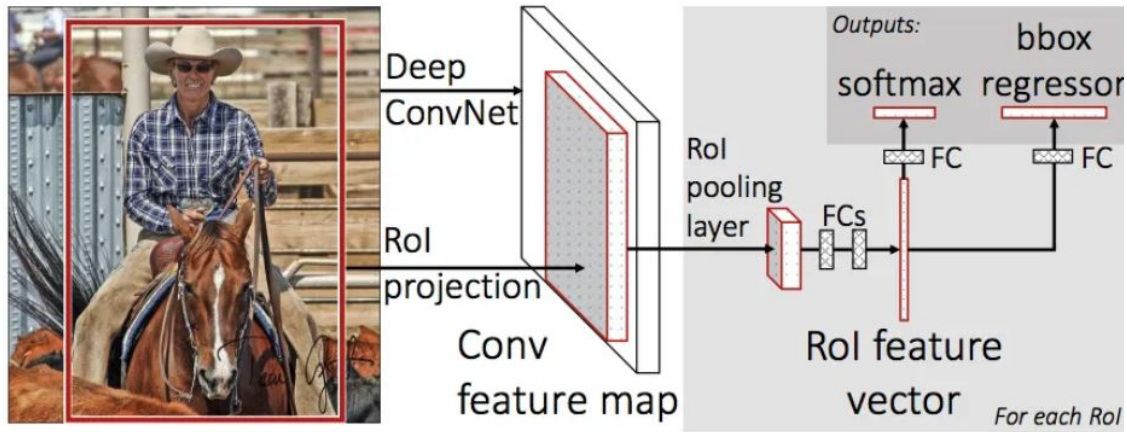
Problem with R-CNN:

- It still takes a huge amount of time to train the network as you would have to classify 2000 region proposals per image.
- It cannot be implemented real time as it takes around 47 seconds for each test image.
- The selective search algorithm is a fixed algorithm. Therefore, no learning is happening at that stage. This could lead to the generation of bad candidate region proposals.

Fast-RCNN Process:

- The same author of the previous paper(R-CNN) solved some of the drawbacks of R-CNN to build a faster object detection algorithm and it was called Fast R-CNN.
- The approach is similar to the R-CNN algorithm.
- But, instead of feeding the region proposals to the CNN, we feed the input image to the CNN to generate a convolutional feature map.
- From the convolutional feature map, we identify the region of proposals and warp them into squares
- By using a RoI pooling layer we reshape them into a fixed size so that it can be fed into a fully connected layer.
- From the RoI feature vector, we use a softmax layer to predict the class of the proposed region and also the offset values for the bounding box.

Fast RCNN

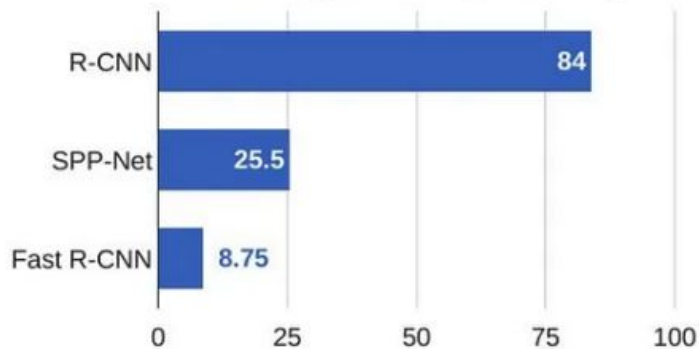


Fast R-CNN

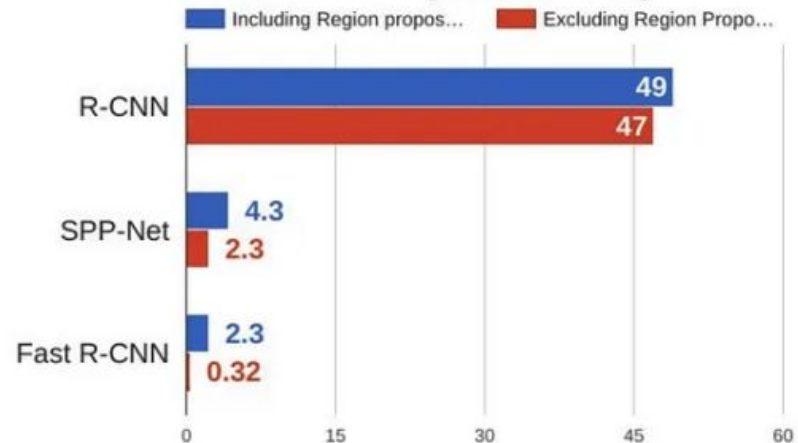
- Run CNN once per image, not per region
- Steps:
 - Input full image to CNN(e.g. VGG16)
 - Get convolutional feature map
 - Use ROI Pooling to extract fixed-size features from proposals
 - Apply Fully connected layers
 - Use Softmax classifier + Bounding box regressor
- Improvement: shared computation, faster training

Comparison of object detection algorithms

Training time (Hours)



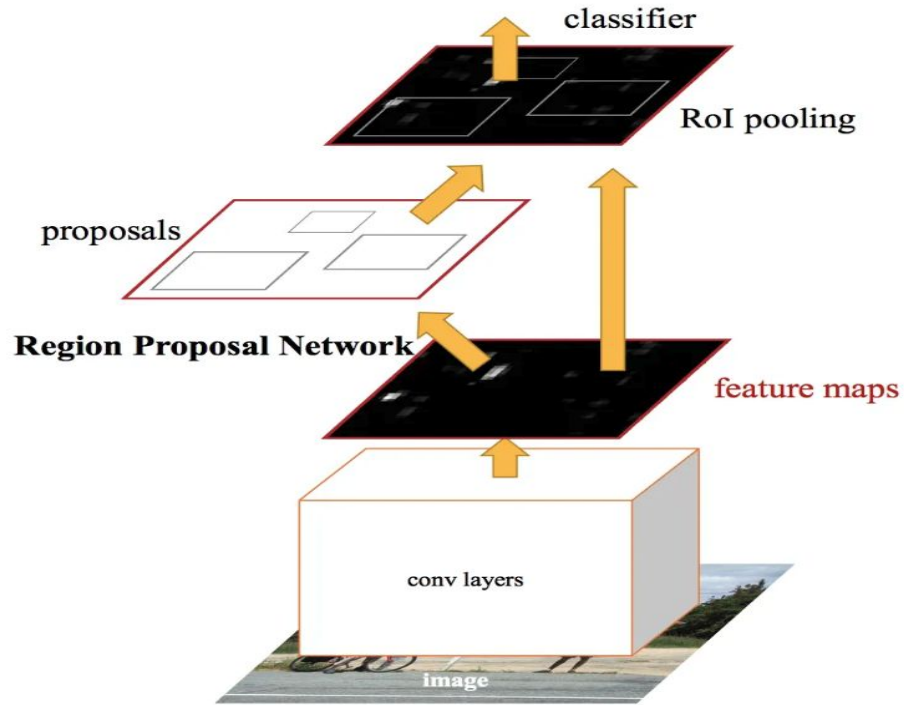
Test time (seconds)



Fast R-CNN Architecture Diagram:

- Input full image
- ConvNet -> Feature Map
- ROI Pooling for each proposal
- FC layers
 - Softmax +BBox Regressor

Faster R-CNN



Faster R-CNN

Faster R-CNN

- Key innovation: Replace Selective Search with Region Proposal Network(RPN)
- Pipeline:
 - Convnet for feature maps
 - RPN generate region proposals
 - ROI Pooling from shared feature maps
 - Classification +BBox Regression
- Benefit: Fully end-to-end
- Much faster (~5fps)

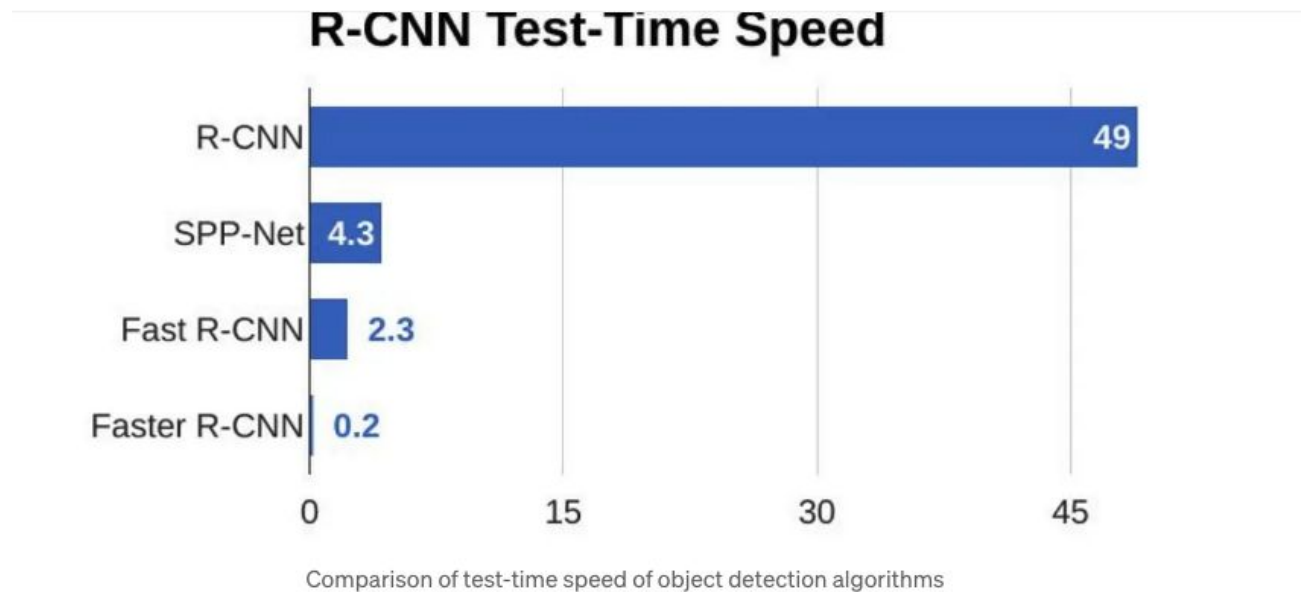
Faster R-CNN

- Similar to Fast R-CNN, the image is provided as an input to a convolutional network which provides a convolutional feature map.
- Instead of using selective search algorithm on the feature map to identify the region proposals, a separate network is used to predict the region proposals.
- The predicted region proposals are then reshaped using a RoI pooling layer.
- Then it used to classify the image within the proposed region and predict the offset values for the bounding boxes.

Faster R-CNN Architecture Diagram

- Input Image
- Conv layers -> Shared Feature map
- RPN -> proposals
- ROI Pooling
- FC layers
- Classifier +Regressor

Comparison of test-time speed of object detection



Comparison table:

Metric	R-CNN	Fast R-CNN	Faster R-CNN
Feature Sharing	No	Yes	Yes
Region Proposals	Selective Search	Selective Search	RPN
Speed	Slow	Faster	Real-Time-ish
Training	Multi-stage	Single-stage	

Key insight:

- R-CNN: High accuracy, low speed
- Fast R-CNN: Speed through shared computation
- Faster R-CNN: End-to-end, near real-time
- Introduced the concept of **RPN** — a major innovation in detection

Yolo Algorithm: You Look Only Once

- Overview of Object Detection:
 - Detecting what objects are in the image and where they are(class of the object, position of the object)
- Key terms output: Bounding Boxes, class labels, confidence score
- Traditional Algorithms: R-CNN family(Selective Search-> R-CNN-> Fast-R-CNN -> Faster R-CNN)
- Motivation: **YOLO is faster and works in real time.**

What is Yolo?

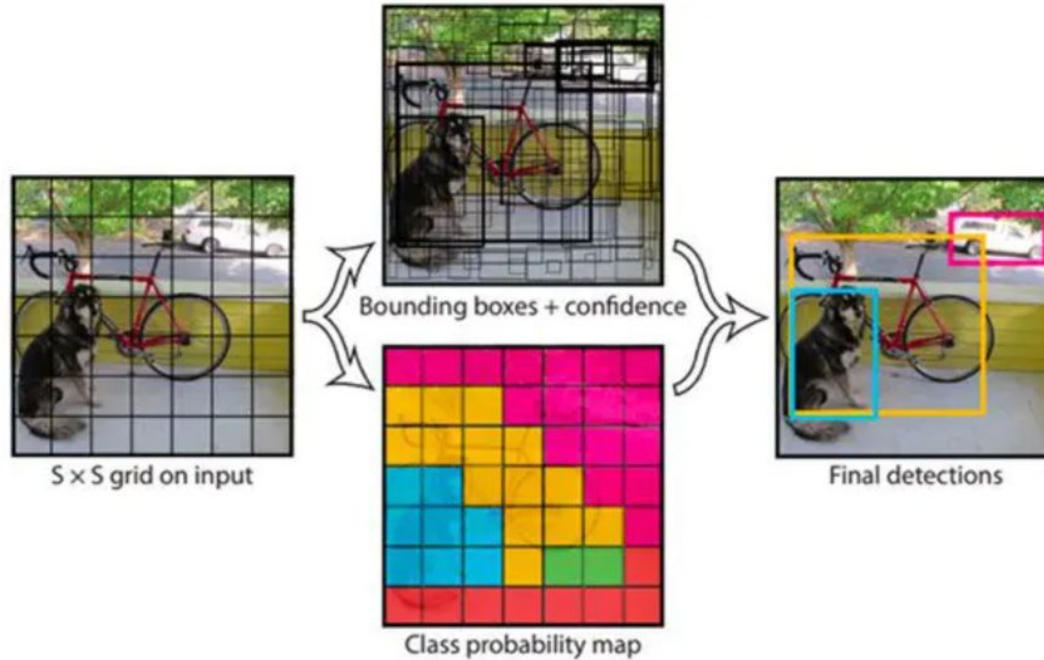
- Yolo stands for You Only Look Once
- It is a single-stage object Detection algorithm.
- Unlike two-stage detectors(e.g., R-CNN), Yolo does detection and classification in one forward pass.
- It is designated for speed and efficiency, making it suitable for real time applications.

How Yolo Works? Step-by-step

- ***Grid-Based Detection:***
 - Image is divided into an $S \times S$ grid (e.g., 7×7)
 - Each grid cell is responsible for predicting:
 - B bounding boxes (e.g., 2 per cell)
 - Confidence Scores for those boxes
 - C class probabilities.
- ***Bounding Box Prediction:***
- Each box predictions:
 - (x,y)-center coordinates (relative to the grid cell)
 - (w, h)-width and height (relative to the image)
 - Confidence Score = $\text{Pr}(\text{Object}) * \text{IoU}(\text{pred}, \text{truth})$

Loss function in Yolo

- Yolo loss function is sum of three components:
 - Localization loss(squared error for (x,y,w,h))
 - Confidence loss: for object/no-object prediction
 - Classification loss: cross-entropy over class labels



Backbone: A convolutional neural network (e.g., Darknet in YOLOv1-v4).

The final layer outputs a tensor of shape:

$$S \times S \times (B * 5 + C)$$

- $B * 5$: for each box (x, y, w, h, confidence)
- C : class probabilities

Yolo strength and limitation:

- Strengths:
 - Very fast(real-time)
 - Global context(no-pass)
 - Simpler pipeline
- Limitations:
 - Struggle with small objects
 - Grid cell may detect only one object
 - Localization error for overlapping objects

Demos and Tools:

- [YOLOv8 \(Ultralytics\)](#)
- Try on [Roboflow](#)
- Use [Google Colab](#) to run object detection live

•
`from ultralytics import YOLO`

```
model = YOLO("yolov5s.pt") # or yolov8s.pt
results = model("image.jpg") # inference
results.show()
```

Yolo Versions:

Version	Improvements
v1	Original version
v2 (YOLO9000)	Better resolution, anchor boxes
v3	Multi-scale predictions, residual blocks
v4	Improved backbone (CSPDarknet53), better mAP
v5	PyTorch implementation (unofficial)
v6/7/8	Speed, accuracy, model scaling improvements
YOLO-NAS	Neural architecture search
YOLOv9 (2024)	Unified visual representation with GELAN

- Image Classification (Object Recognition)
- Image Retrieval
- Object Detection
- Image Segmentation
- Video Classification
- Activity Recognition
- Pose Estimation
- Playing Video (e.g. Atari) Games
- Image Captioning
- Generative Models: Variational-Autoencoders, GANs, Diffusion Models

To the Student: Search for some of these tasks in Google Image Search.

See Also: <https://huggingface.co/models>